■■■■ - ■■■■■

■■■■■■■■■

■■■■■: 31047 ■
■■: ■30■ + ■30■ (■60■)

■■30■■

```
// ========================================
// ■■■■ - ■■■■■
// ■■■■■■■■■■
// ========================================


// ========== ■■: src/app/api/admin/add-reading-progress-field/route.ts ==========

import { createClient } from '@/lib/supabase/server'
import { NextResponse } from 'next/server'

export async function GET() {
  const supabase = await createClient()

  try {
    // ■■ SQL ■■■■
    const { error } = await (supabase as any).rpc('exec_sql', {
      sql: `
        ALTER TABLE user_book_preferences
        ADD COLUMN IF NOT EXISTS last_reading_progress JSONB;

        COMMENT ON COLUMN user_book_preferences.last_reading_progress IS '■■■■■■■■■■■■■■■■■■■■■■■';

        CREATE INDEX IF NOT EXISTS idx_user_book_preferences_reading_progress_book_id
        ON user_book_preferences ((last_reading_progress->>'bookId'))
        WHERE last_reading_progress IS NOT NULL;
      `
    })

    if (error) {
      // ■■ exec_sql ■■■■■■■■■■ SQL
      const { error: directError } = await supabase
        .from('user_book_preferences')
        .select('last_reading_progress')
        .limit(1)

      if (directError && directError.message.includes('column')) {
        return NextResponse.json({
          success: false,
          error: '■■■■■■■■■■ Supabase ■■■■■■■■■■',
          sql: `
-- ■ Supabase SQL Editor ■■■■

ALTER TABLE user_book_preferences
ADD COLUMN IF NOT EXISTS last_reading_progress JSONB;

COMMENT ON COLUMN user_book_preferences.last_reading_progress IS '■■■■■■■■■■■■■■■■■■■■■■■';

CREATE INDEX IF NOT EXISTS idx_user_book_preferences_reading_progress_book_id
ON user_book_preferences ((last_reading_progress->>'bookId'))
```

```
WHERE last_reading_progress IS NOT NULL;
          `
        }, { status: 500 })
      }

      return NextResponse.json({
        success: true,
        message: '■■■■■■■■■■■■',
        existingError: error.message
      })
    }

    return NextResponse.json({
      success: true,
      message: '■■■■■■■'
    })
  } catch (error: any) {
    return NextResponse.json({
      success: false,
      error: error.message,
      sql: `
-- ■ Supabase SQL Editor ■■■■

ALTER TABLE user_book_preferences
ADD COLUMN IF NOT EXISTS last_reading_progress JSONB;

COMMENT ON COLUMN user_book_preferences.last_reading_progress IS '■■■■■■■■■■■■■■■■■■■■■■■■';

CREATE INDEX IF NOT EXISTS idx_user_book_preferences_reading_progress_book_id
ON user_book_preferences ((last_reading_progress->>'bookId'))
WHERE last_reading_progress IS NOT NULL;
      `
    }, { status: 500 })
  }
}


// ========== ■■: src/app/api/admin/apply-migration/route.ts ==========

import { createClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'
import { createClient as createAdminClient } from '@supabase/supabase-js'

/**
 * ■■■■■ Migration
 * POST /api/admin/apply-migration
 *
 * ■■■■ API ■■ admin ■■■■ SQL■■■■■■■■■■■■
 * ■■■■■■■■ Supabase Dashboard ■ migration ■■■■
 */
```

```
export async function POST(request: NextRequest) {
  try {
    // ■■■■■■■
    const supabase = await createClient()
    const { data: { user } } = await supabase.auth.getUser()

    if (!user) {
      return NextResponse.json({
        success: false,
        error: 'Unauthorized - Please login first'
      }, { status: 401 })
    }

    const body = await request.json()
    const { sql } = body

    if (!sql) {
      return NextResponse.json({
        success: false,
        error: 'Missing SQL parameter'
      }, { status: 400 })
    }

    console.log('[Admin] Applying migration...')

    // ■■ admin client ■■ SQL
    // ■■■■■■■■■■■■■ SUPABASE_SERVICE_ROLE_KEY
    const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL
    const supabaseServiceKey = process.env.SUPABASE_SERVICE_ROLE_KEY

    if (!supabaseUrl || !supabaseServiceKey) {
      return NextResponse.json({
        success: false,
        error: 'Missing Supabase configuration'
      }, { status: 500 })
    }

    const adminClient = createAdminClient(supabaseUrl, supabaseServiceKey)

    // ■■ SQL - ■■ rpc ■■■■■■■■■■■ wrapper ■■■
    // ■■■■■■■ SQL■■■■■■ Dashboard ■■
    console.log('[Admin] SQL prepared (length:', sql.length, 'chars)')

    return NextResponse.json({
      success: true,
      message: 'SQL prepared successfully. Please execute it in Supabase Dashboard.',
      data: {
        sql: sql,
        instructions: [
          '1. Open Supabase Dashboard: https://supabase.com/dashboard',
```

```
          '2. Select your project',
          '3. Go to SQL Editor',
          '4. Create a new query',
          '5. Paste the SQL below and execute',
          '6. Check for errors in the output'
        ],
        appliedAt: new Date().toISOString(),
        user: user.email
      }
    })

  } catch (error) {
    console.error('[Admin] Exception preparing migration:', error)
    return NextResponse.json({
      success: false,
      error: error instanceof Error ? error.message : 'Unknown error',
      details: error
    }, { status: 500 })
  }
}


// ========== ■■: src/app/api/admin/apply-scope-stats-migration/route.ts ==========

import { createClient } from '@supabase/supabase-js'
import { NextRequest, NextResponse } from 'next/server'

/**
 * POST /api/admin/apply-scope-stats-migration
 * ■■ get_book_scope_stats ■■■■
 * ■■■■■
 */
export async function POST(request: NextRequest) {
  try {
    // ■■ service role key ■■ admin ■■■
    const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL
    const supabaseServiceKey = process.env.SUPABASE_SERVICE_ROLE_KEY

    if (!supabaseUrl || !supabaseServiceKey) {
      return NextResponse.json({
        success: false,
        error: '■■ Supabase ■■'
      }, { status: 500 })
    }

    const supabaseAdmin = createClient(supabaseUrl, supabaseServiceKey, {
      auth: {
        autoRefreshToken: false,
        persistSession: false
      }
```

```
    })

    // ■■ SQL
    const migrationSQL = `
-- ■■■■■■■■■■scope■■
CREATE OR REPLACE FUNCTION get_book_scope_stats(
  p_book_id UUID,
  p_total_words INTEGER
)
RETURNS TABLE (
  all INTEGER,
  unknown INTEGER,
  fuzzy INTEGER,
  new INTEGER,
  known INTEGER,
  mistakes INTEGER
) AS $$
BEGIN
  RETURN QUERY
  WITH
  -- ■■■■■■■■■■
  progress_counts AS (
    SELECT
      status,
      COUNT(*) as count
    FROM word_progress
    WHERE book_id = p_book_id
    GROUP BY status
  ),
  -- ■■■■■■■■■
  mistake_count AS (
    SELECT COUNT(*) as count
    FROM mistakes
    WHERE book_id = p_book_id
  )
  SELECT
    p_total_words as all,
    COALESCE((SELECT count FROM progress_counts WHERE status = 'unknown'), 0) as unknown,
    COALESCE((SELECT count FROM progress_counts WHERE status = 'fuzzy'), 0) as fuzzy,
    GREATEST(0, p_total_words - COALESCE((SELECT count FROM progress_counts WHERE status = 'unknown'), 0)
                              - COALESCE((SELECT count FROM progress_counts WHERE status = 'fuzzy'), 0)
                              - COALESCE((SELECT count FROM progress_counts WHERE status = 'known'), 0)) as new,
    COALESCE((SELECT count FROM progress_counts WHERE status = 'known'), 0) as known,
    COALESCE((SELECT count FROM mistake_count), 0) as mistakes;
END;
$$ LANGUAGE plpgsql SECURITY DEFINER;

-- ■■■■
COMMENT ON FUNCTION get_book_scope_stats IS '■■■■■■■■■■■■■■■■■■■■all, unknown, fuzzy, new, known, mistakes■
```

```
-- ■■
GRANT EXECUTE ON FUNCTION get_book_scope_stats TO authenticated;
    `.trim()

    // ■■ SQL - ■■ PostgreSQL ■ pg_catalog.exec_sql
    const { data, error } = await supabaseAdmin
      .from('pg_catalog')
      .select('*')
      .limit(1)

    // ■■ Supabase JS ■■■■■■■■■■■■ SQL■
    // ■■■■■■■■■ SQL ■■■■■■■■■■
    // ■■■■■ pg_stat_statements ■■

    // ■■■■■■■■■■
    const { data: existingFunction, error: checkError } = await supabaseAdmin
      .rpc('get_book_scope_stats', {
        p_book_id: '00000000-0000-0000-0000-000000000000',
        p_total_words: 0
      })

    if (checkError && checkError.message.includes('function get_book_scope_stats')) {
      // ■■■■■■■■■ SQL ■■■■■■
      return NextResponse.json({
        success: false,
        needsManualExecution: true,
        message: '■■■■■■■■■■■■■■■ SQL',
        sql: migrationSQL,
        instructions: [
          '1. ■■ Supabase Dashboard: https://app.supabase.com',
          '2. ■■■■■■■',
          '3. ■■ SQL Editor',
          '4. ■■■■■■■■ SQL'
        ]
      })
    }

    // ■■■■■■■■■■■■■
    return NextResponse.json({
      success: true,
      message: 'get_book_scope_stats ■■■■■■',
      note: '■■■■■■■■■■■■■■■■■■■ SQL'
    })

  } catch (error) {
    console.error('[Migration] Error:', error)
    return NextResponse.json({
      success: false,
      error: error instanceof Error ? error.message : '■■■■■'
    }, { status: 500 })
```

```
  }
}

/**
 * GET /api/admin/apply-scope-stats-migration
 * ■■■■■■■■■■
 */
export async function GET(request: NextRequest) {
  try {
    const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL
    const supabaseServiceKey = process.env.SUPABASE_SERVICE_ROLE_KEY

    if (!supabaseUrl || !supabaseServiceKey) {
      return NextResponse.json({
        success: false,
        error: '■■ Supabase ■■'
      }, { status: 500 })
    }

    const supabaseAdmin = createClient(supabaseUrl, supabaseServiceKey)

    // ■■■■■■■■■■■■■■
    const { data, error } = await supabaseAdmin
      .rpc('get_book_scope_stats', {
        p_book_id: '00000000-0000-0000-0000-000000000000',
        p_total_words: 0
      })

    if (error) {
      return NextResponse.json({
        success: false,
        exists: false,
        error: error.message
      })
    }

    return NextResponse.json({
      success: true,
      exists: true,
      message: 'get_book_scope_stats ■■■■■■■■■■■■'
    })

  } catch (error) {
    return NextResponse.json({
      success: false,
      exists: false,
      error: error instanceof Error ? error.message : '■■■■'
    })
  }
}
```

```
351
352
353 // ========== ■■: src/app/api/admin/auth/logout/route.ts ==========
354
355 /**
356  * ■■■■■ API
357  */
358
359 import { createClient } from '@/lib/supabase/server'
360 import { cookies } from 'next/headers'
361 import { logAdminAction } from '@/lib/admin-auth'
362 import { NextResponse } from 'next/server'
363
364 export async function POST() {
365   try {
366     const supabase = await createClient()
367
368     // ■■■■■■
369     await logAdminAction('admin_logout')
370
371     // ■■■■
372     await supabase.auth.signOut()
373
374     return NextResponse.json({ success: true })
375   } catch (error) {
376     console.error('Admin logout error:', error)
377     return NextResponse.json(
378       { success: false, error: '■■■■' },
379       { status: 500 }
380     )
381   }
382 }
383
384
385 // ========== ■■: src/app/api/admin/check-code-user/route.ts ==========
386
387 import { createAdminClient } from '@/lib/supabase/server'
388 import { NextResponse } from 'next/server'
389
390 export async function GET(request: Request) {
391   const { searchParams } = new URL(request.url)
392   const code = searchParams.get('code')
393
394   if (!code) {
395     return NextResponse.json({ error: 'Missing code parameter' }, { status: 400 })
396   }
397
398   const supabase = await createAdminClient()
399
400   // 1. ■■■■■■■
```

```
  const { data: invitationCode, error: codeError } = await supabase
    .from('invitation_codes')
    .select('*')
    .eq('code', code)
    .single()

  if (codeError) {
    return NextResponse.json({ error: 'Invitation code not found: ' + codeError.message }, { status: 404 })
  }

  // 2. ■■■■■■■■■■■■■■■■■■■■■10■■
  const { data: recentUsers, error: usersError } = await supabase
    .from('users')
    .select('*')
    .order('created_at', { ascending: false })
    .limit(10)

  if (usersError) {
    return NextResponse.json({ error: usersError.message }, { status: 500 })
  }

  // 3. ■■■■■■■■■■■■ID■■■
  const { data: usersWithCode, error: usersWithCodeError } = await supabase
    .from('users')
    .select('*')
    .eq('invitation_code_id', (invitationCode as any).id)

  if (usersWithCodeError) {
    return NextResponse.json({ error: usersWithCodeError.message }, { status: 500 })
  }

  return NextResponse.json({
    invitationCode: {
      id: (invitationCode as any).id,
      code: (invitationCode as any).code,
      package_id: (invitationCode as any).package_id,
      book_permissions: (invitationCode as any).book_permissions,
      feature_permissions: (invitationCode as any).feature_permissions,
      used_count: (invitationCode as any).used_count,
      used_by: (invitationCode as any).used_by
    },
    usersWithThisCode: usersWithCode,
    recentUsers: recentUsers
  })
}


// ========== ■■: src/app/api/admin/check-table-structure/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
```

```
import { NextRequest, NextResponse } from 'next/server'

/**
 * GET /api/admin/check-table-structure
 * ■■ public.users ■■■
 */
export async function GET(request: NextRequest) {
  try {
    const supabase = await createAdminClient()

    // ■■ users ■■■■■
    const { data: columns, error } = await (supabase as any)
      .rpc('query', {
        query: `
          SELECT
            column_name,
            data_type,
            is_nullable,
            column_default
          FROM information_schema.columns
          WHERE table_schema = 'public'
            AND table_name = 'users'
          ORDER BY ordinal_position;
        `
      })

    if (error) {
      // ■■ rpc ■■■■■■■
      const { data: users, error: selectError } = await supabase
        .from('users')
        .select('*')
        .limit(1)

      return NextResponse.json({
        method: 'select *',
        hasData: !!users,
        error: selectError?.message,
        columns: users && users.length > 0 ? Object.keys(users[0]) : [],
        sample: users?.[0]
      })
    }

    return NextResponse.json({
      method: 'information_schema',
      columns
    })

  } catch (error: any) {
    return NextResponse.json({
      error: error.message
```

```
    }, { status: 500 })
  }
}


// ========== ■■: src/app/api/admin/check-user/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'

/**
 * GET /api/admin/check-user?phone=xxx
 * ■■■■■■■■■
 */
export async function GET(request: NextRequest) {
  try {
    const searchParams = request.nextUrl.searchParams
    const phone = searchParams.get('phone')

    if (!phone) {
      return NextResponse.json(
        { error: '■■■■■■■■' },
        { status: 400 }
      )
    }

    const supabase = await createAdminClient()
    const email = `${phone}@phone.xiaoyu.com`

    // 1. ■■ public.users ■
    const { data: publicUser, error: publicError } = await supabase
      .from('users')
      .select('*')
      .eq('phone_number', phone)
      .single()

    // 2. ■■ auth.users
    const { data: { users }, error: authError } = await supabase.auth.admin.listUsers()
    const authUser = (users as any[])?.find((u: any) => u.email === email)

    return NextResponse.json({
      phone,
      email,
      publicUser: publicUser ? {
        exists: true,
        id: (publicUser as any).id,
        email: (publicUser as any).email,
        phone_number: (publicUser as any).phone_number,
        created_at: (publicUser as any).created_at,
        is_banned: (publicUser as any).is_banned,
```

```
        ban_reason: (publicUser as any).ban_reason
      } : {
        exists: false,
        error: publicError?.message
      },
      authUser: authUser ? {
        exists: true,
        id: authUser.id,
        email: authUser.email,
        created_at: authUser.created_at,
        email_confirmed_at: authUser.email_confirmed_at,
        last_sign_in_at: authUser.last_sign_in_at
      } : {
        exists: false
      },
      recommendation: !publicUser && !authUser ? '■■■■' : '■■■■■■■■■■'
    })
  } catch (error: any) {
    console.error('Error checking user:', error)
    return NextResponse.json(
      { error: error.message || 'Internal server error' },
      { status: 500 }
    )
  }
}


// ========== ■■: src/app/api/admin/cleanup-audio-urls/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextResponse } from 'next/server'

/**
 * POST /api/admin/cleanup-audio-urls
 * ■■■■■■■■■■■API URL
 *
 * ■■API■■■■■■■■■ "dict.youdao.com" ■ audio_url
 * ■■■■TTS API■■■■■■■■■■■■■OSS
 */
export async function POST() {
  try {
    const supabase = await createAdminClient()

    console.error('■ ■■■■■■■■■API URL■■■■■SQL■...')

    // ■■■■SQL■■■■■■■■■■■■■■■
    const { data: updateResult, error: updateError } = await (supabase.rpc as any)('exec_sql', {
      sql: `UPDATE words SET audio_url = NULL WHERE audio_url LIKE '%dict.youdao.com%'`
    })

```

```
601      // ■■ RPC ■■■■■■■■■■■■■■■■
602      if (updateError) {
603        console.warn('■■  RPC■■■■■■■■■■...')
604        console.error('■ ■■■■:', updateError)
605        return NextResponse.json(
606          { error: '■■■■■', details: updateError.message, hint: '■■■■■■■■■■■■SQL' },
607          { status: 500 }
608        )
609      }
610
611      console.error('■ ■■■■■')
612
613      // ■■■■■■■■URL■■
614      const { count, error: countError } = await supabase
615        .from('words')
616        .select('id', { count: 'exact', head: true })
617        .like('audio_url', '%dict.youdao.com%')
618
619      const remainingCount = countError ? 0 : (count || 0)
620
621      return NextResponse.json({
622        success: true,
623        message: `■■■■■■■■ ${remainingCount} ■■■URL`,
624        remainingCount
625      })
626
627    } catch (error) {
628      console.error('■ ■■audio_url■■:', error)
629      return NextResponse.json(
630        { error: '■■■■■■■■', details: error.message },
631        { status: 500 }
632      )
633    }
634  }
635
636  /**
637   * GET /api/admin/cleanup-audio-urls
638   * ■■■■■■■■■■■■URL■■■■
639   */
640  export async function GET() {
641    try {
642      const supabase = await createAdminClient()
643
644      const { count, error } = await supabase
645        .from('words')
646        .select('id', { count: 'exact', head: true })
647        .like('audio_url', '%dict.youdao.com%')
648
649      if (error) {
650        return NextResponse.json(
```

```
        { error: '[redacted]', details: error },
        { status: 500 }
      )
    }

    return NextResponse.json({
      success: true,
      wrongUrlCount: count || 0,
      message: `[redacted] ${count || 0} [redacted]API URL[redacted]`
    })

  } catch (error) {
    console.error('[redacted]:', error)
    return NextResponse.json(
      { error: '[redacted]', details: error.message },
      { status: 500 }
    )
  }
}


// ========== [redacted]: src/app/api/admin/cleanup-orphaned-book/route.ts ==========

import { createClient, getCurrentUser } from '@/lib/supabase/server'
import { NextResponse } from 'next/server'

/**
 * POST /api/admin/cleanup-orphaned-book
 * [redacted]
 */
export async function POST() {
  const user = await getCurrentUser()

  if (!user) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
  }

  try {
    const supabase = await createClient()
    const orphanedBookId = '68090a64-edc6-4145-bde3-6cd56997868f'

    // 1. [redacted] user_book_preferences
    const { error: prefsError } = await supabase
      .from('user_book_preferences')
      .delete()
      .eq('book_id', orphanedBookId)

    if (prefsError) {
      console.error('[redacted] user_book_preferences [redacted]:', prefsError)
    }
```

```

    // 2. ■■ books
    const { error: bookError } = await supabase
      .from('books')
      .delete()
      .eq('id', orphanedBookId)

    if (bookError) {
      console.error('■■ books ■■:', bookError)
      return NextResponse.json({
        success: false,
        error: bookError.message
      }, { status: 500 })
    }

    return NextResponse.json({
      success: true,
      message: '■■■■■■■■'
    })
  } catch (error) {
    console.error('■■■■■■■■■:', error)
    return NextResponse.json(
      { error: error instanceof Error ? error.message : '■■■■■■' },
      { status: 500 }
    )
  }
}


// ========== ■■: src/app/api/admin/clear-rate-limit/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'

/**
 * POST /api/admin/clear-rate-limit
 * ■■■■■■■■■
 * ■■: { "ipAddress": "■■■■■IP■■■■■■■■■" }
 */
export async function POST(request: NextRequest) {
  try {
    const body = await request.json()
    const { ipAddress } = body

    const supabase = await createAdminClient()

    let deletedCount = 0
    let message = ''

    if (ipAddress) {
```

```
751       // ■■■■IP■■■
752       const { data: regData, error: regError } = await supabase
753         .from('registration_attempts')
754         .delete()
755         .eq('ip_address', ipAddress)
756         .select()
757
758       const { data: invData, error: invError } = await supabase
759         .from('invitation_code_attempts')
760         .delete()
761         .eq('ip_address', ipAddress)
762         .select()
763
764       if (regError) console.error('Error deleting registration_attempts:', regError)
765       if (invError) console.error('Error deleting invitation_code_attempts:', invError)
766
767       const regCount = regData?.length || 0
768       const invCount = invData?.length || 0
769       deletedCount = regCount + invCount
770
771       message = `■■■ IP ${ipAddress} ■■■■■■■■: ${regCount}■■■■■■■■: ${invCount}■■`
772     } else {
773       // ■■■■■■■
774       const { count: regCount } = await supabase
775         .from('registration_attempts')
776         .delete()
777         .neq('ip_address', '000000000000')
778
779       const { count: invCount } = await supabase
780         .from('invitation_code_attempts')
781         .delete()
782         .neq('code', '000000000000')
783
784       deletedCount = (regCount || 0) + (invCount || 0)
785       message = `■■■■■■■■■■■■■■■■: ${regCount}■■■■■■■■: ${invCount}■■`
786     }
787
788     return NextResponse.json({
789       success: true,
790       message,
791       deletedCount
792     })
793   } catch (error: any) {
794     console.error('Error in POST /api/admin/clear-rate-limit:', error)
795     return NextResponse.json(
796       { error: 'Internal server error', details: error.message },
797       { status: 500 }
798     )
799   }
800 }
```

```

// ========== ■■: src/app/api/admin/clear-users/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'

/**
 * DELETE /api/admin/clear-users
 * ■■■■■■■■■■■■■■■■■■■
 * ■■ ■■■■
 */
export async function DELETE(request: NextRequest) {
  try {
    const supabase = await createAdminClient()

    // ■■■■■■■■■
    const { data: { user } } = await supabase.auth.getUser()
    if (!user?.email) {
      return NextResponse.json({ error: 'Not authenticated' }, { status: 401 })
    }

    const adminEmail = user.email

    // ■■■■■■■■■■■■
    const { data: { users }, error } = await supabase.auth.admin.listUsers()

    if (error) {
      return NextResponse.json({ error: error.message }, { status: 500 })
    }

    let deletedCount = 0
    for (const u of users) {
      // ■■■■■■■■
      if (u.email === adminEmail) {
        console.log('■ Keeping admin:', u.email)
        continue
      }

      // ■■■■■■■
      await supabase.auth.admin.deleteUser(u.id)
      deletedCount++
      console.log('■■ Deleted user:', u.email)
    }

    return NextResponse.json({
      success: true,
      message: `■■■ ${deletedCount} ■■■■■■`,
      adminEmail,
      deletedCount
```

```
    })
  } catch (error: any) {
    console.error('Error:', error)
    return NextResponse.json(
      { error: error.message || 'Internal server error' },
      { status: 500 }
    )
  }
}


// ========== ■■: src/app/api/admin/delete-test-users/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'

/**
 * DELETE /api/admin/delete-test-users
 * ■■■■■■■■■■■■■■■■■■■■
 * ■■ ■■■■■■■■■■■■■■■■■■■■■■
 */
export async function DELETE(request: NextRequest) {
  try {
    // ■■ admin ■■■
    const supabase = await createAdminClient()

    // ■■ RPC ■■■■■■■■
    const { data, error } = await (supabase as any).rpc('delete_test_users')

    if (error) {
      console.error('Error deleting test users:', error)
      return NextResponse.json(
        { error: 'Failed to delete test users', details: error.message },
        { status: 500 }
      )
    }

    return NextResponse.json({
      success: true,
      message: '■■■■■■■■■■■■■■■',
      data
    })
  } catch (error: any) {
    console.error('Error in DELETE /api/admin/delete-test-users:', error)
    return NextResponse.json(
      { error: 'Internal server error', details: error.message },
      { status: 500 }
    )
  }
}
```

```


// ========== ■■: src/app/api/admin/diagnose-auth/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'

/**
 * GET /api/admin/diagnose-auth
 * ■■ Supabase Auth ■■
 */
export async function GET(request: NextRequest) {
  const diagnostics: any = {
    timestamp: new Date().toISOString(),
    checks: []
  }

  try {
    const supabase = await createAdminClient()

    // ■■ 1: ■■■■
    diagnostics.checks.push({
      name: 'Supabase ■■■■',
      status: 'running'
    })

    try {
      const { data, error } = await supabase.from('users').select('count').limit(1)
      diagnostics.checks[0].status = error ? 'failed' : 'passed'
      diagnostics.checks[0].error = error?.message
      diagnostics.checks[0].data = data
    } catch (err: any) {
      diagnostics.checks[0].status = 'failed'
      diagnostics.checks[0].error = err.message
    }

    // ■■ 2: Auth Admin API ■■
    diagnostics.checks.push({
      name: 'Auth Admin API ■■',
      status: 'running'
    })

    try {
      const { data: { users }, error } = await supabase.auth.admin.listUsers()
      diagnostics.checks[1].status = error ? 'failed' : 'passed'
      diagnostics.checks[1].error = error?.message
      diagnostics.checks[1].userCount = users?.length || 0
    } catch (err: any) {
      diagnostics.checks[1].status = 'failed'
      diagnostics.checks[1].error = err.message
```

```
    }

    // ■■ 3: ■■■■5■■■
    diagnostics.checks.push({
      name: '■■ Auth ■■',
      status: 'running'
    })

    try {
      const { data: { users }, error } = await supabase.auth.admin.listUsers()
      if (!error && users) {
        const recentUsers = (users as any)
          .sort((a: any, b: any) => new Date(b.created_at).getTime() - new Date(a.created_at).getTime())
          .slice(0, 5)
          .map((u: any) => ({
            id: u.id,
            email: u.email,
            created_at: u.created_at,
            confirmed: !!u.email_confirmed_at
          }))
        diagnostics.checks[2].status = 'passed'
        diagnostics.checks[2].users = recentUsers
      } else {
        diagnostics.checks[2].status = 'failed'
        diagnostics.checks[2].error = error?.message
      }
    } catch (err: any) {
      diagnostics.checks[2].status = 'failed'
      diagnostics.checks[2].error = err.message
    }

    // ■■ 4: ■■■■■■■
    diagnostics.checks.push({
      name: '■■■■■■■',
      status: 'running',
      testEmail: `test-${Date.now()}@test.com`
    })

    try {
      const testEmail = `test-${Date.now()}@test.com`
      const { data, error } = await supabase.auth.admin.createUser({
        email: testEmail,
        password: 'test123456',
        email_confirm: true
      })

      if (error) {
        diagnostics.checks[3].status = 'failed'
        diagnostics.checks[3].error = error.message
        diagnostics.checks[3].details = error
```

```
      } else if (data.user) {
        diagnostics.checks[3].status = 'passed'
        diagnostics.checks[3].createdUserId = data.user.id

        // [redacted]
        await supabase.auth.admin.deleteUser(data.user.id)
        diagnostics.checks[3].cleanup = 'deleted'
      }
    } catch (err: any) {
      diagnostics.checks[3].status = 'failed'
      diagnostics.checks[3].error = err.message
      diagnostics.checks[3].stack = err.stack
    }

    // [redacted] 5: [redacted]
    diagnostics.checks.push({
      name: '[redacted]',
      status: 'passed',
      supabaseUrl: process.env.NEXT_PUBLIC_SUPABASE_URL ? 'set' : 'missing',
      hasServiceKey: process.env.SUPABASE_SERVICE_ROLE_KEY ? 'yes' : 'no'
    })

    return NextResponse.json(diagnostics)

  } catch (error: any) {
    diagnostics.error = error.message
    diagnostics.stack = error.stack
    return NextResponse.json(diagnostics, { status: 500 })
  }
}


// ========== [redacted]: src/app/api/admin/find-user/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextResponse } from 'next/server'

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url)
  const phone = searchParams.get('phone')

  if (!phone) {
    return NextResponse.json({ error: 'Missing phone parameter' }, { status: 400 })
  }

  const supabase = await createAdminClient()

  // [redacted]
  const { data: users, error: userError } = await supabase
    .from('users')
```

```
    .select('*')
    .eq('phone_number', phone)
    .order('created_at', { ascending: false })

  if (userError) {
    return NextResponse.json({ error: 'Query failed: ' + userError.message }, { status: 500 })
  }

  if (!users || users.length === 0) {
    return NextResponse.json({ error: 'User not found' }, { status: 404 })
  }

  return NextResponse.json({
    users,
    count: users.length
  })
}


// ========== ■■: src/app/api/admin/fix-invitation-function/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextResponse } from 'next/server'

export async function POST() {
  const supabase = await createAdminClient()

  // SQL to fix the use_invitation_code function
  const sql = `
    CREATE OR REPLACE FUNCTION use_invitation_code(code_param TEXT, user_id_param UUID)
    RETURNS BOOLEAN AS $$
    DECLARE
      invitation_code RECORD;
      package_record RECORD;
    BEGIN
      -- ■■■■■
      SELECT * INTO invitation_code
      FROM invitation_codes
      WHERE code = code_param
      AND is_active = true
      AND used_by IS NULL
      AND (expires_at IS NULL OR expires_at > NOW())
      FOR UPDATE;

      -- ■■■■■■■■■■■■
      IF NOT FOUND THEN
        RETURN false;
      END IF;

      -- ■■■■■■■■■
```

```
1101        UPDATE invitation_codes
1102        SET
1103          used_by = user_id_param,
1104          used_at = NOW(),
1105          used_count = used_count + 1
1106        WHERE id = invitation_code.id;
1107
1108        -- ■■■■■■■■■■■■■■
1109        SELECT * INTO package_record
1110        FROM invitation_packages
1111        WHERE id = invitation_code.package_id;
1112
1113        -- ■■■■■■■
1114        -- ■■■■■■■■validity_days■■■■■■■■■■■■■■■■validity_days
1115        UPDATE users
1116        SET
1117          feature_permissions = invitation_code.feature_permissions,
1118          book_permissions = invitation_code.book_permissions,
1119          invitation_code_id = invitation_code.id,
1120          permission_expires_at = CASE
1121            -- ■■■■■■■■■■validity_days■■■■■■■validity_days
1122            WHEN package_record.id IS NOT NULL AND package_record.validity_days IS NOT NULL
1123            THEN NOW() + (package_record.validity_days || ' days')::INTERVAL
1124            -- ■■■■■■■validity_days■null■■■null■■■■■
1125            WHEN package_record.id IS NOT NULL AND package_record.validity_days IS NULL
1126            THEN NULL
1127            -- ■■■■■■■■■■■■■■validity_days
1128            WHEN invitation_code.validity_days IS NOT NULL
1129            THEN NOW() + (invitation_code.validity_days || ' days')::INTERVAL
1130            -- ■■■■■■■null■■■■■
1131            ELSE NULL
1132          END
1133        WHERE id = user_id_param;
1134
1135        RETURN true;
1136      END;
1137      $$ LANGUAGE plpgsql;
1138
1139      -- ■■■■■
1140      COMMENT ON FUNCTION use_invitation_code IS '■■■■■■■■■■■■■■■■■■■validity_days■■■';
1141    `
1142
1143    // Execute SQL using RPC (if you have exec_sql function available)
1144    // Otherwise, you'll need to run this SQL directly in Supabase dashboard
1145    const { data, error } = await (supabase as any).rpc('exec_sql', { sql_query: sql })
1146
1147    if (error) {
1148      console.error('Migration error:', error)
1149      return NextResponse.json({
1150        error: error.message,
```

```
      note: 'Please run this SQL manually in Supabase SQL Editor'
    }, { status: 500 })
  }

  return NextResponse.json({
    success: true,
    message: 'Function fixed successfully',
    data
  })
}

// GET endpoint to check invitation code details
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url)
  const code = searchParams.get('code')

  if (!code) {
    return NextResponse.json({ error: 'Missing code parameter' }, { status: 400 })
  }

  const supabase = await createAdminClient()

  // Query invitation code details
  const { data: invitationCode, error: codeError } = await supabase
    .from('invitation_codes')
    .select(`
      *,
      package:invitation_packages(*)
    `)
    .eq('code', code)
    .single()

  if (codeError || !invitationCode) {
    return NextResponse.json({ error: codeError?.message || 'Invitation code not found' }, { status: 500 })
  }

  // Query users who used this invitation code
  const { data: users, error: usersError } = await supabase
    .from('users')
    .select('*')
    .eq('invitation_code_id', (invitationCode as any).id)

  if (usersError) {
    return NextResponse.json({ error: usersError.message }, { status: 500 })
  }

  return NextResponse.json({
    invitationCode: invitationCode || null,
    users: users || [],
    package: (invitationCode as any)?.package || null
```

```
  })
}


// ========== ■■: src/app/api/admin/fix-users-table/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'

/**
 * POST /api/admin/fix-users-table
 * ■■ public.users ■■■ updated_at ■■■■■
 */
export async function POST(request: NextRequest) {
  try {
    const supabase = await createAdminClient()

    // ■■ SQL ■■
    const sql = `
      -- ■■ updated_at ■■
      ALTER TABLE public.users
      ADD COLUMN IF NOT EXISTS updated_at TIMESTAMPTZ DEFAULT NOW();

      -- ■■■■■■■■■■
      CREATE OR REPLACE FUNCTION public.handle_updated_at()
      RETURNS TRIGGER AS $$
      BEGIN
        NEW.updated_at = NOW();
        RETURN NEW;
      END;
      $$ LANGUAGE plpgsql;

      -- ■■■■■
      DROP TRIGGER IF EXISTS set_updated_at ON public.users;
      CREATE TRIGGER set_updated_at
        BEFORE UPDATE ON public.users
        FOR EACH ROW
        EXECUTE FUNCTION public.handle_updated_at();
    `

    // ■■ rpc ■■■■ SQL
    const { data, error } = await (supabase as any).rpc('exec_sql', { sql_query: sql })

    if (error) {
      // ■■ rpc ■■■■■■■■ SQL ■■■■■■■■
      return NextResponse.json({
        success: false,
        message: '■■■■■■■■ SQL',
        sql,
        error: error.message
```

```
      })
    }

    return NextResponse.json({
      success: true,
      message: '██████users ████ updated_at ██'
    })

  } catch (error: any) {
    return NextResponse.json({
      success: false,
      error: error.message,
      hint: '██ Supabase Dashboard ■ SQL Editor █████ SQL'
    }, { status: 500 })
  }
}


// ========== ██: src/app/api/admin/force-create-user/route.ts ==========

import { createAdminClient } from '@/lib/supabase/server'
import { NextRequest, NextResponse } from 'next/server'

/**
 * POST /api/admin/force-create-user
 * ██ Admin API ████████ Auth ███
 */
export async function POST(request: NextRequest) {
  const logs: string[] = []

  try {
    const body = await request.json()
    const { phone, password, invitationCode } = body

    logs.push(`■ ███: ${phone}`)
    logs.push(`■ ███: ${invitationCode}`)

    const supabase = await createAdminClient()
    const email = `${phone}@phone.xiaoyu.com`

    // Step 1: ████
    logs.push(`\n■ Step 1: ████...`)
    const { data: codeData, error: codeError } = await supabase
      .from('invitation_codes')
      .select('*')
      .eq('code', invitationCode)
      .eq('is_active', true)
      .single()

    if (codeError || !codeData) {
```

```
        logs.push(`■ ■■■■■■: ${codeError?.message}`)
        return NextResponse.json({
          success: false,
          error: '■■■■■■■■■■',
          logs
        })
      }
      logs.push(`■ ■■■■■■`)

      // Step 2: ■■■■■■■■■■
      logs.push(`\n■ Step 2: ■■■■■■■■■■...`)
      const { data: existingUser } = await supabase
        .from('users')
        .select('id')
        .eq('phone_number', phone)
        .single()

      if (existingUser) {
        logs.push(`■ ■■■■■■: id=${(existingUser as any).id}`)
        return NextResponse.json({
          success: false,
          error: '■■■■■■■■■■■■■■■',
          logs
        })
      }
      logs.push(`■ ■■■■■■■■■■■`)

      // Step 3: ■■ Admin API ■■■■■■■■■■ Auth ■■■■
      logs.push(`\n■ Step 3: ■■ Admin API ■■■■■...`)
      logs.push(`■ ■■: ${email}`)

      const { data: authData, error: authError } = await supabase.auth.admin.createUser({
        email,
        password,
        email_confirm: true, // ■■■■■■■
        user_metadata: {
          phone_number: phone
        }
      })

      if (authError) {
        logs.push(`■ Admin API ■■■■■■■: ${authError.message}`)
        logs.push(`■■■■■: ${JSON.stringify(authError)}`)

        // ■■■■■■■■■■■■■■■■■■■
        if (authError.message.includes('already exists') || authError.message.includes('duplicate')) {
          logs.push(`\n■■ ■■■■■■■ Auth■■■■■■■■■■...`)

          // ■■■■■■■■■■■■■■■■
          const { data: { users }, error: listError } = await supabase.auth.admin.listUsers()
```

```
1351
1352          if (!listError && users) {
1353            const existingAuthUser = (users as any).find((u: any) => u.email === email)
1354            if (existingAuthUser) {
1355              logs.push(`■ ■■■■ Auth ■■: id=${existingAuthUser.id}`)
1356
1357              // ■■■ public.users
1358              const { error: syncError } = await (supabase.from('users') as any).insert({
1359                id: existingAuthUser.id,
1360                email: email,
1361                phone_number: phone,
1362                full_name: existingAuthUser.user_metadata?.full_name || existingAuthUser.user_metadata?.name || nu
1363                avatar_url: existingAuthUser.user_metadata?.avatar_url || null,
1364                metadata: { invitation_code_used: invitationCode }
1365              })
1366
1367              if (syncError) {
1368                logs.push(`■ ■■■■: ${syncError.message}`)
1369              } else {
1370                logs.push(`■ ■■■■`)
1371              }
1372
1373              return NextResponse.json({
1374                success: true,
1375                message: '■■■■■■■■■■■■■■',
1376                userId: existingAuthUser.id,
1377                logs
1378              })
1379            }
1380          }
1381        }
1382
1383        return NextResponse.json({
1384          success: false,
1385          error: `■■■■■■■: ${authError.message}`,
1386          logs
1387        })
1388      }
1389
1390      if (!authData.user) {
1391        logs.push(`■ Admin API ■■■■■■`)
1392        return NextResponse.json({
1393          success: false,
1394          error: '■■■■■■■■■■■■■■■■■■■',
1395          logs
1396        })
1397      }
1398
1399      logs.push(`■ Admin API ■■■■■■■: id=${authData.user.id}`)
1400
```

```
    // Step 4: ■■■ public.users
    logs.push(`\n■ Step 4: ■■■ public.users...`)
    const { error: dbError } = await (supabase.from('users') as any).insert({
      id: authData.user.id,
      email: email,
      phone_number: phone,
      full_name: authData.user.user_metadata?.full_name || authData.user.user_metadata?.name || null,
      avatar_url: authData.user.user_metadata?.avatar_url || null,
      metadata: { invitation_code_used: invitationCode }
    })

    if (dbError) {
      logs.push(`■ ■■■ public.users ■■: ${dbError.message}`)

      // ■■■■■ Auth ■■
      await supabase.auth.admin.deleteUser(authData.user.id)
      logs.push(`■■ ■■■ Auth ■■`)

      return NextResponse.json({
        success: false,
        error: '■■■■■■■■■■■■',
        logs
      })
    }

    logs.push(`■ ■■■ public.users ■■`)

    // Step 5: ■■■■■■■
    logs.push(`\n■ Step 5: ■■■■■■■...`)
    const { error: quotaError } = await (supabase.from('user_quotas') as any).insert({
      user_id: authData.user.id,
      daily_smart_import_limit: 500,
      daily_smart_import_used: 0
    })

    if (quotaError) {
      logs.push(`■■ ■■■■■■■■■■■■: ${quotaError.message}`)
    } else {
      logs.push(`■ ■■■■■■■`)
    }

    // Step 6: ■■■■■
    logs.push(`\n■ Step 6: ■■■■■...`)
    const { data: useCodeResult, error: useCodeError } = await (supabase as any).rpc('use_invitation_code', {
      code_param: invitationCode,
      user_id_param: authData.user.id
    })

    if (useCodeError || !useCodeResult) {
      logs.push(`■■ ■■■■■■■■■■■■: ${useCodeError?.message}`)
```

```
    } else {
      logs.push(`■ ■■■■■■■■`)
    }

    logs.push(`\n■ ■■■■■■`)
    logs.push(`\n■ ■■■■■■■■■■■■`)
    logs.push(`■■■■: ${phone}`)
    logs.push(`■■: (■■■■■■■)`)

    return NextResponse.json({
      success: true,
      message: '■■■■■■■■■■■■■■■',
      userId: authData.user.id,
      email: authData.user.email,
      logs
    })

  } catch (error: any) {
    logs.push(`\n■ ■■: ${error.message}`)
    logs.push(`■■: ${error.stack}`)
    return NextResponse.json({
      success: false,
      error: error.message || '■■■■■■■■■',
      logs
    }, { status: 500 })
  }
}


// ========== ■■: src/app/api/admin/invitation-codes/create/route.ts ==========

/**
 * ■■■■■■ API
 * POST /api/admin/invitation-codes/create
 */

import { createAdminClient } from '@/lib/supabase/server'
import { requireAdmin } from '@/lib/admin-auth'
import { logAdminAction } from '@/lib/admin-auth'
import { NextRequest, NextResponse } from 'next/server'

/**
 * ■■■■■■■■
 */
function generateInvitationCode(): string {
  const chars = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789' // ■■■■■■■■
  let code = ''
  for (let i = 0; i < 8; i++) {
    if (i > 0 && i % 4 === 0) code += '-'
    code += chars.charAt(Math.floor(Math.random() * chars.length))
```

... ■■■■ 28047 ■ ...

■■30■■

```
 * - Middleware
 *
 * Key differences from client.ts:
 * - Uses createServerClient (not createBrowserClient)
 * - Has access to server-side cookies
 * - Can be used in async server components
 * - Supports SSR with proper authentication
 */

import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
import type { Database } from '@/types/database'

// ■■■■■■■■
let proxyFetch: typeof fetch | undefined

async function getProxyFetch(): Promise<typeof fetch | undefined> {
  if (process.env.NODE_ENV !== 'development') {
    return undefined
  }

  if (proxyFetch) {
    return proxyFetch
  }

  const proxyUrl = process.env.HTTPS_PROXY || process.env.HTTP_PROXY || 'http://127.0.0.1:7890'

  try {
    const { ProxyAgent, fetch: undiciFetch } = await import('undici')
    const proxyAgent = new ProxyAgent(proxyUrl)

    proxyFetch = async (input: RequestInfo | URL, init?: RequestInit) => {
      // ■■ headers - ■■■ Headers ■■■■■■■■
      let headers: Record<string, string> = {}
      if (init?.headers) {
        if (init.headers instanceof Headers) {
          init.headers.forEach((value, key) => {
            headers[key] = value
          })
        } else if (Array.isArray(init.headers)) {
          init.headers.forEach(([key, value]) => {
            headers[key] = value
          })
        } else {
          headers = { ...init.headers as Record<string, string> }
        }
      }

      const url = typeof input === 'string' ? input : input instanceof URL ? input.href : (input as Request).ur
```

```
29598       const response = await undiciFetch(url, {
29599         method: init?.method,
29600         headers: headers,
29601         body: init?.body as any,
29602         dispatcher: proxyAgent,
29603       })
29604
29605       return response as any
29606     }
29607
29608     return proxyFetch
29609   } catch (error) {
29610     console.error('[Supabase] Failed to configure proxy fetch:', error)
29611     return undefined
29612   }
29613 }
29614
29615 /**
29616  * Creates a Supabase client for server-side usage
29617  *
29618  * @example
29619  * Example usage:
29620  * - Server Components: await createClient()
29621  * - Server Actions: await createClient()
29622  * - Route Handlers: await createClient()
29623  */
29624 export async function createClient() {
29625   const cookieStore = await cookies()
29626
29627   // ■ Fix: ■■■■■ HTTPS ■■
29628   // ■ Next.js server component ■■■■■■■■■■■■■■■■■■■■
29629   const isHttps = process.env.NODE_ENV === 'production'
29630
29631   // ■ Debug: ■■■■■■■■■■■■■■■■
29632   // const isDev = process.env.NODE_ENV === 'development'
29633
29634   // ■ ■■■■■■■■■■■■■■
29635   const isDev = false
29636
29637   /* // ■■■■■■■■■■■■■■■
29638   if (isDev) {
29639     const allCookies = cookieStore.getAll()
29640     const sbCookies = allCookies.filter(c => c.name.includes('sb-'))
29641
29642     console.log('■ [Server Client] Cookie info:', {
29643       total: allCookies.length,
29644       sbCookies: sbCookies.map(c => ({
29645         name: c.name,
29646         hasValue: !!c.value,
29647         valueLength: c.value?.length || 0,
```

```
      })),
      isHttps
    })
}
*/

// ■■■■■■■■■ fetch
const customFetch = await getProxyFetch()

return createServerClient<Database>(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
  {
    global: {
      fetch: customFetch || undefined,
    },
    cookies: {
      get(name: string) {
        const value = cookieStore.get(name)?.value

        /* // ■■■■■■■■■■■■■■■■
        if (isDev && name.includes('sb-')) {
          console.log(`■ [Server Client] Reading cookie: ${name}, found: ${!!value}`)
        }
        */

        return value
      },
      set(name: string, value: string, options: any) {
        try {
          // ■ Fix: ■■■■■■■■■ secure ■■
          cookieStore.set({
            name,
            value,
            ...options,
            secure: isHttps,  // HTTPS ■■■■■■ true■HTTP ■■■■ false
            sameSite: 'lax',
            httpOnly: true,
          })

          /* // ■■■■■■■■■■■■■■■■
          if (isDev) {
            console.log('[createClient] Set cookie:', name, 'secure:', isHttps)
          }
          */
        } catch (error) {
          console.error('[createClient] Failed to set cookie:', name, 'error:', error)
        }
      },
      remove(name: string, options: any) {
```

```
          try {
            /* // ■■■■■■■■■■■■■■■
            if (isDev && name.includes('sb-')) {
              console.log('■ [createClient] Attempting to remove cookie:', name)
            }
            */

            // ■ ■■■■■■■ auth-token ■■■ cookies
            if (name.includes('auth-token')) {
              /* // ■■■■■■■■■■■■■■■■
              if (isDev) {
                console.log('■ [createClient] BLOCKED removal of auth cookie:', name)
              }
              */
              return  // ■■■■
            }

            cookieStore.set({
              name,
              value: '',
              ...options,
              secure: isHttps,  // ■■■■■■■■
            })
          } catch (error) {
            console.error('[createClient] Failed to remove cookie:', name, 'error:', error)
          }
        },
      },
    }
  )
}

/**
 * Create admin client that bypasses RLS using service role key
 * WARNING: Only use for admin operations that need to bypass RLS
 */
export async function createAdminClient() {
  const { createClient: createDirectClient } = await import('@supabase/supabase-js')

  // ■■■■■■■■■ fetch
  const customFetch = await getProxyFetch()

  return createDirectClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.SUPABASE_SERVICE_ROLE_KEY || process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      ...(customFetch ? { global: { fetch: customFetch } } : {}),
      auth: {
        autoRefreshToken: false,
        persistSession: false
```

```
      }
    }
  )
}

/**
 * Alias for createClient - use in Server Actions for better code clarity
 *
 * @example
 * ```ts
 * // app/actions/books.ts
 * 'use server'
 *
 * import { createClient as createSupabaseClient } from '@/lib/supabase/server'
 *
 * export async function getBooks() {
 *   const supabase = await createSupabaseClient()
 *   const { data, error } = await supabase.from('books').select('*')
 *   return { data, error }
 * }
 * ```
 */
export { createClient as createClientForActions }

/**
 * Get the currently logged-in user from Supabase Auth
 *
 * @returns {Promise<User | null>} The user object if logged in, null otherwise
 *
 * @example
 * ```ts
 * import { getCurrentUser } from '@/lib/supabase/server'
 *
 * const user = await getCurrentUser()
 * if (!user) {
 *   redirect('/login')
 * }
 * ```
 */
export async function getCurrentUser() {
  /* // ■■■■■■■■■■■■■■■
  const isDev = process.env.NODE_ENV === 'development'

  if (isDev) {
    console.log('■ [getCurrentUser] Starting...')
  }
  */

  const supabase = await createClient()

```

```
  const {
    data: { user },
    error,
  } = await supabase.auth.getUser()

  /* // ■■■■■■■■■■■■■■■
  if (isDev) {
    console.log('■ [getCurrentUser] Result:', {
      hasUser: !!user,
      userId: user?.id,
      error: error?.message
    })
  }
  */

  if (error || !user) {
    return null
  }

  return user
}

/**
 * Check if user is authenticated
 *
 * @returns {Promise<boolean>} True if user is logged in
 *
 * @example
 * ```ts
 * import { isAuthenticated } from '@/lib/supabase/server'
 *
 * if (!(await isAuthenticated())) {
 *   return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
 * }
 * ```
 */
export async function isAuthenticated() {
  const user = await getCurrentUser()
  return !!user
}

/**
 * Get user profile from the users table (includes custom fields)
 *
 * @returns {Promise<any>} User profile if logged in, null otherwise
 *
 * @example
 * ```ts
 * import { getUserProfile } from '@/lib/supabase/server'
 *
```

```
 * const profile = await getUserProfile()
 * if (profile) {
 *   console.log('Phone:', profile.phone_number)
 * }
 * ```
 */
export async function getUserProfile() {
  const user = await getCurrentUser()

  if (!user) {
    return null
  }

  const supabase = await createClient()

  const { data, error } = await supabase
    .from('users')
    .select('*')
    .eq('id', user.id)
    .single()

  if (error) {
    console.error('Failed to fetch user profile:', error)
    return null
  }

  return data
}

/**
 * Require authentication - throws error if user is not logged in
 *
 * @returns {Promise<User>} The authenticated user
 * @throws {Error} If user is not authenticated
 *
 * @example
 * ```ts
 * import { requireAuth } from '@/lib/supabase/server'
 *
 * export default async function ProtectedPage() {
 *   const user = await requireAuth()
 *   // User is guaranteed to be logged in here
 * }
 * ```
 */
export async function requireAuth() {
  const user = await getCurrentUser()

  if (!user) {
    throw new Error('Authentication required')
```

```
  }

  return user
}


// ========== ■■: src/lib/timeout.ts ==========

/**
 * ■■■■■■
 * ■■■■API■■■■■■■■■■■■■
 */

/**
 * ■Promise■■■■■■
 * @param promise ■■■■Promise
 * @param timeoutMs ■■■■■■■■
 * @param errorMessage ■■■■■■
 * @returns Promise■■■■■
 */
export async function withTimeout<T>(
  promise: Promise<T>,
  timeoutMs: number,
  errorMessage: string = 'Operation timed out'
): Promise<T> {
  // ■■■■Promise
  const timeoutPromise = new Promise<never>((_, reject) => {
    setTimeout(() => {
      reject(new Error(`${errorMessage} (timeout: ${timeoutMs}ms)`))
    }, timeoutMs)
  })

  // ■■Promise.race■■■■■■■■■■■■
  return Promise.race([promise, timeoutPromise])
}

/**
 * ■Promise.all■■■■■■
 * @param promises Promise■■
 * @param timeoutMs ■■■■■■■■
 * @param errorMessage ■■■■■■
 * @returns ■■Promise■■■■■■■■
 */
export async function withTimeoutAll<T>(
  promises: Promise<T>[],
  timeoutMs: number,
  errorMessage: string = 'Batch operation timed out'
): Promise<T[]> {
  return withTimeout(Promise.all(promises), timeoutMs, errorMessage)
}
```

```
29948
29949  /**
29950   * ■■■■■■■AbortController
29951   * ■■■■■■■fetch■■
29952   * @param timeoutMs ■■■■■■■■
29953   * @returns AbortController
29954   */
29955  export function createTimeoutController(timeoutMs: number): AbortController {
29956    const controller = new AbortController()
29957    setTimeout(() => controller.abort(), timeoutMs)
29958    return controller
29959  }
29960
29961  /**
29962   * ■■■JSON■■■■■■■■■■■■
29963   * @param request Next.js Request■■
29964   * @param options ■■■■■
29965   * @returns ■■■■JSON■■
29966   */
29967  export async function safeJsonParse<T = any>(
29968    request: Request,
29969    options: {
29970      timeout?: number // ■■■■■■■■■■■■5000
29971      maxSize?: number // ■■■■■■■■■■■■■■■1MB
29972    } = {}
29973  ): Promise<T> {
29974    const { timeout = 5000, maxSize = 1024 * 1024 } = options
29975
29976    // ■■Content-Length
29977    const contentLength = request.headers.get('content-length')
29978    if (contentLength) {
29979      const size = parseInt(contentLength, 10)
29980      if (size > maxSize) {
29981        throw new Error(`Request body too large: ${size} bytes (max: ${maxSize})`)
29982      }
29983    }
29984
29985    // ■■■■■■
29986    return withTimeout(
29987      request.json(),
29988      timeout,
29989      'JSON parse timeout'
29990    )
29991  }
29992
29993  /**
29994   * ■■■■■■■■■■■■■■■■
29995   * @param queryFn ■■■■■■■■
29996   * @param options ■■■■■
29997   * @returns ■■■■
```

```
 */
export async function safeDbQuery<T>(
  queryFn: () => Promise<T>,
  options: {
    timeout?: number // ■■■■■■■■■■■10000
    retries?: number // ■■■■■■■■0
    retryDelay?: number // ■■■■■■■■■■■100
  } = {}
): Promise<T> {
  const { timeout = 10000, retries = 0, retryDelay = 100 } = options

  let lastError: Error | null = null

  for (let attempt = 0; attempt <= retries; attempt++) {
    try {
      return await withTimeout(queryFn(), timeout, `Database query timeout (attempt ${attempt + 1})`)
    } catch (error) {
      lastError = error as Error

      // ■■■■■■■■■■■■■■■
      if (attempt < retries && lastError.message.includes('timeout')) {
        console.log(`■■ Query timeout, retrying... (${attempt + 1}/${retries})`)
        await new Promise(resolve => setTimeout(resolve, retryDelay * (attempt + 1)))
        continue
      }

      throw lastError
    }
  }

  throw lastError
}

/**
 * ■■■■■
 * @param ms ■■■■■■■■■
 */
export function delay(ms: number): Promise<void> {
  return new Promise(resolve => setTimeout(resolve, ms))
}

/**
 * ■■■■■■■■■
 * ■■■■■■■■■
 * @param fn ■■■■■■■■■false■■■■■■
 * @param options ■■■■■
 */
export async function safeLoop(
  fn: () => Promise<boolean>,
  options: {
```

```
    maxIterations?: number // ■■■■■■■■■100
    timeout?: number // ■■■■■■■■■■■■30000
    iterationDelay?: number // ■■■■■■■■■■■■■■0
  } = {}
): Promise<void> {
  const { maxIterations = 100, timeout = 30000, iterationDelay = 0 } = options

  const startTime = Date.now()
  let iterations = 0

  while (iterations < maxIterations) {
    // ■■■■■■
    if (Date.now() - startTime > timeout) {
      throw new Error(`Loop timeout after ${iterations} iterations (timeout: ${timeout}ms)`)
    }

    const shouldContinue = await fn()
    if (!shouldContinue) {
      break
    }

    iterations++

    // ■■■■■
    if (iterationDelay > 0) {
      await delay(iterationDelay)
    }
  }

  if (iterations >= maxIterations) {
    console.warn(`■■ Loop reached max iterations (${maxIterations})`)
  }
}


// ========== ■■: src/lib/timeUtils.ts ==========

/**
 * ■■■■■■■■
 * ■■■■■■■■■■■■■■■■■■■■
 */

/**
 * ■■■■■■■■■■■
 */
const TIME_UNITS = {
  minute: 60 * 1000,
  hour: 60 * 60 * 1000,
  day: 24 * 60 * 60 * 1000,
  week: 7 * 24 * 60 * 60 * 1000,
```

```
  month: 30 * 24 * 60 * 60 * 1000,
  year: 365 * 24 * 60 * 60 * 1000
} as const

/**
 * ■■■■■■■■■■■■■■■■■
 *
 * @param timestamp - ■■■■■■■■
 * @returns ■■■■■■■■■"■■"■"2■■■"■"3■■"■■
 *
 * @example
 * formatTimeAgo(Date.now() - 30 * 1000) // '■■'
 * formatTimeAgo(Date.now() - 5 * 60 * 1000) // '5■■■'
 * formatTimeAgo(Date.now() - 3 * 60 * 60 * 1000) // '3■■■'
 * formatTimeAgo(Date.now() - 2 * 24 * 60 * 60 * 1000) // '2■■'
 * formatTimeAgo(Date.now() - 7 * 24 * 60 * 60 * 1000) // '1■■'
 */
export function formatTimeAgo(timestamp: number): string {
  // ■■■■
  if (!timestamp || typeof timestamp !== 'number') {
    console.warn('[formatTimeAgo] Invalid timestamp:', timestamp)
    return '■■■■'
  }

  const now = Date.now()
  const diff = now - timestamp

  // ■■■■■■■■■■■■■■
  if (diff < 0) {
    console.warn('[formatTimeAgo] Timestamp is in the future:', timestamp)
    return '■■'
  }

  // ■■1■■■■■
  if (diff < TIME_UNITS.minute) {
    return '■■'
  }

  // ■■1■■■X■■■
  if (diff < TIME_UNITS.hour) {
    const minutes = Math.floor(diff / TIME_UNITS.minute)
    return `${minutes}■■■`
  }

  // ■■1■■X■■■
  if (diff < TIME_UNITS.day) {
    const hours = Math.floor(diff / TIME_UNITS.hour)
    return `${hours}■■■`
  }

```

```
  // ■■1■■X■■
  if (diff < TIME_UNITS.week) {
    const days = Math.floor(diff / TIME_UNITS.day)
    return `${days}■■`
  }

  // ■■1■■■X■■
  if (diff < TIME_UNITS.month) {
    const weeks = Math.floor(diff / TIME_UNITS.week)
    return `${weeks}■■`
  }

  // ■■1■■X■■■
  if (diff < TIME_UNITS.year) {
    const months = Math.floor(diff / TIME_UNITS.month)
    return `${months}■■`
  }

  // ■■1■■X■■
  const years = Math.floor(diff / TIME_UNITS.year)
  return `${years}■■`
}

/**
 * ■■■■■■■■■■■■■■■■■■■■
 *
 * @param timestamp - ■■■■■■■■
 * @returns ■■■■■■■■"2024-01-10 15:30"■
 *
 * @example
 * formatDateTime(Date.now() - 3 * 24 * 60 * 60 * 1000)
 * // ■■: '2024-01-07 15:30' (■■■■■2024-01-10)
 */
export function formatDateTime(timestamp: number): string {
  if (!timestamp || typeof timestamp !== 'number') {
    console.warn('[formatDateTime] Invalid timestamp:', timestamp)
    return '■■■■'
  }

  const date = new Date(timestamp)

  // ■■■■ YYYY-MM-DD HH:mm
  const year = date.getFullYear()
  const month = String(date.getMonth() + 1).padStart(2, '0')
  const day = String(date.getDate()).padStart(2, '0')
  const hours = String(date.getHours()).padStart(2, '0')
  const minutes = String(date.getMinutes()).padStart(2, '0')

  return `${year}-${month}-${day} ${hours}:${minutes}`
}
```

```

// ========== ■■: src/lib/urlValidation.ts ==========

/**
 * URL■■■■■■
 * ■■■■■■URL■■■■■■■■■■■■
 */

import { ScopeType } from '@/types/progress'

/**
 * ■■■■■■■■■■
 */
const VALID_SCOPES: ScopeType[] = ['all', 'unknown', 'fuzzy', 'known', 'new']

/**
 * ■■■■■■■■■
 * @param scope - URL■■■■scope■
 * @returns ■■■ScopeType■■■■■■'unknown'■■■■■■
 *
 * @example
 * validateScope('all') // 'all'
 * validateScope('invalid') // 'unknown'
 * validateScope('') // 'unknown'
 * validateScope(undefined) // 'unknown'
 */
export function validateScope(scope: string | null | undefined): ScopeType {
  // ■■■■■■■■undefined/null/■■■■■
  if (!scope || typeof scope !== 'string') {
    return 'unknown' // ■■■■
  }

  // ■■■■■■■■■■■■■5■■■■■
  if (VALID_SCOPES.includes(scope as ScopeType)) {
    return scope as ScopeType
  }

  // ■■■■■■■■■
  return 'unknown'
}

/**
 * ■■hash■■■■■
 * @param hash - URL hash■■■■ '#word-10'■
 * @returns ■■■■■■■■■0-based■■■■■■■■undefined
 *
 * @example
 * validateHashIndex('#word-10') // 10
 * validateHashIndex('word-5') // 5
```

```
 * validateHashIndex('#word-abc') // undefined
 * validateHashIndex('') // undefined
 * validateHashIndex(undefined) // undefined
 */
export function validateHashIndex(hash: string | null | undefined): number | undefined {
  // ■■■■■■■■■undefined/null/■■■■■
  if (!hash || typeof hash !== 'string') {
    return undefined
  }

  // ■■■■■■■■■■ '#word-10' ■ 'word-5' ■■■■■
  const match = hash.match(/word-(\d+)/)
  if (!match) {
    return undefined
  }

  // ■■■■■■
  const index = parseInt(match[1], 10)

  // ■■■■■■■■■■■■■■■
  if (isNaN(index) || index < 0) {
    return undefined
  }

  return index
}

/**
 * ■■■■■■URL■■■■■■■■
 * @param value - URL■■■■■■■■■
 * @param defaultValue - ■■■■■■■■■■■■■■■
 * @param min - ■■■■■■■■
 * @param max - ■■■■■■■■
 * @returns ■■■■■■■■■■■■
 *
 * @example
 * safeGetInt('10', 0) // 10
 * safeGetInt('abc', 0) // 0
 * safeGetInt('5', 0, 1, 100) // 5
 * safeGetInt('0', 0, 1, 100) // 1 (■■min■■■min)
 * safeGetInt('150', 0, 1, 100) // 100 (■■max■■■max)
 */
export function safeGetInt(
  value: string | null | undefined,
  defaultValue: number,
  min?: number,
  max?: number
): number {
  // ■■■■■■■■■undefined/null
  if (value === null || value === undefined) {
```

```
    return defaultValue
  }

  // ■■■■■
  const num = parseInt(value, 10)

  // ■■■■■
  if (isNaN(num)) {
    return defaultValue
  }

  // ■■■■■■
  let result = num
  if (min !== undefined && result < min) {
    result = min
  }
  if (max !== undefined && result > max) {
    result = max
  }

  return result
}


// ========== ■■: src/lib/utils.ts ==========

import { clsx, type ClassValue } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

/**
 * ■■■■■■■■■■■■
 */
export function formatDate(dateString: string): string {
  const date = new Date(dateString)
  const now = new Date()
  const diff = now.getTime() - date.getTime()

  // ■■24■■■■■■■■■■
  if (diff < 24 * 60 * 60 * 1000) {
    const hours = Math.floor(diff / (60 * 60 * 1000))
    if (hours < 1) {
      const minutes = Math.floor(diff / (60 * 1000))
      return minutes < 1 ? '■■' : `${minutes}■■■`
    }
    return `${hours}■■■`
  }
```

```

  // ■■7■
  if (diff < 7 * 24 * 60 * 60 * 1000) {
    const days = Math.floor(diff / (24 * 60 * 60 * 1000))
    return `${days}■■`
  }

  // ■■■■■■■■
  return date.toLocaleDateString('zh-CN', {
    year: 'numeric',
    month: '2-digit',
    day: '2-digit'
  })
}


// ========== ■■: src/lib/utils/cache.ts ==========

/**
 * Redis ■■■■
 * ■■■■■■■■■■■API■■■
 */

import { Redis } from 'ioredis'

let redisClient: Redis | null = null

/**
 * ■■ Redis ■■■
 */
export async function getRedisClient(): Promise<Redis> {
  if (!redisClient) {
    const redisUrl = process.env.REDIS_URL

    if (!redisUrl) {
      console.warn('REDIS_URL not configured, cache disabled')
      throw new Error('Redis not configured')
    }

    redisClient = new Redis(redisUrl, {
      maxRetriesPerRequest: 3,
      retryStrategy: (times) => {
        const delay = Math.min(times * 50, 2000)
        return delay
      }
    })

    redisClient.on('error', (err) => {
      console.error('Redis Client Error:', err)
      redisClient = null
```

```
    })
  }

  return redisClient
}

/**
 * ■■■■■■■
 * @param word ■■
 * @param data ■■■■
 * @param ttl ■■■■■■■■■■■30■
 */
export async function cacheWordData(
  word: string,
  data: any,
  ttl: number = 30 * 24 * 3600 // 30■
): Promise<void> {
  try {
    const redis = await getRedisClient()
    const key = `word:data:${word.toLowerCase()}`
    await redis.setex(key, ttl, JSON.stringify(data))
    console.log(`[Cache] Cached word: ${word}`)
  } catch (error) {
    console.error('[Cache] Failed to cache word:', error)
    // ■■■■■■■■■■■
  }
}

/**
 * ■■■■■■■■■■
 * @param word ■■
 * @returns ■■■■■■null
 */
export async function getCachedWordData(
  word: string
): Promise<any | null> {
  try {
    const redis = await getRedisClient()
    const key = `word:data:${word.toLowerCase()}`
    const data = await redis.get(key)

    if (data) {
      console.log(`[Cache] Hit for word: ${word}`)
      return JSON.parse(data)
    }

    console.log(`[Cache] Miss for word: ${word}`)
    return null
  } catch (error) {
    console.error('[Cache] Failed to get cached word:', error)
```

```
    return null
  }
}

/**
 * ■■■■
 * @param pattern ■■■■■■■■■■■■■
 */
export async function invalidateCache(pattern: string): Promise<void> {
  try {
    const redis = await getRedisClient()
    const keys = await redis.keys(pattern)

    if (keys.length > 0) {
      await redis.del(...keys)
      console.log(`[Cache] Invalidated ${keys.length} keys matching: ${pattern}`)
    }
  } catch (error) {
    console.error('[Cache] Failed to invalidate cache:', error)
  }
}

/**
 * ■■ Redis ■■■■
 */
export async function isRedisAvailable(): Promise<boolean> {
  try {
    const redis = await getRedisClient()
    await redis.ping()
    return true
  } catch (error) {
    console.warn('[Cache] Redis not available:', error)
    return false
  }
}


// ========== ■■: src/lib/utils/retry.ts ==========

/**
 * ■■■■■ - ■■■■■■■■
 * @param fn ■■■■■■■
 * @param maxRetries ■■■■■■■
 * @returns Promise<T>
 */
export async function retryWithBackoff<T>(
  fn: () => Promise<T>,
  maxRetries: number = 3
): Promise<T> {
  for (let attempt = 0; attempt < maxRetries; attempt++) {
```

```
    try {
      return await fn()
    } catch (error: any) {
      // ■■■■■■■■■■■■■■
      if (attempt === maxRetries - 1) {
        throw error
      }

      // ■■■■429■■■■■■■■■■■■■■■
      if (error.status === 429 || error.message?.includes('429')) {
        const delay = Math.pow(2, attempt) * 1000 // 1s, 2s, 4s
        console.log(`Retry attempt ${attempt + 1}/${maxRetries} after ${delay}ms`)
        await sleep(delay)
      } else {
        // ■■■■■■■■■■■■■■
        throw error
      }
    }
  }

  throw new Error('Max retries exceeded')
}

/**
 * ■■■■■
 * @param ms ■■■■■■
 */
export function sleep(ms: number): Promise<void> {
  return new Promise(resolve => setTimeout(resolve, ms))
}

/**
 * ■■■■API■■■■■■■■■■■■■■
 */
export function parseYoudaoResponse(data: any, word: string) {
  try {
    const simple = data.simple?.word?.[0]
    const ec = data.ec?.word?.[0]
    const ee = data.ee?.word?.[0]
    const blng = data.blng_sents_part?.['sentence-pair']
    const phrs = data.phrs?.phrs
    const syno = data.syno?.synos

    // ■■■■■■■■■■■■■
    const ukPhonetic = simple?.ukphone || simple?.phone || ''
    const usPhonetic = simple?.usphone || simple?.phone || ''
    const phonetic = usPhonetic || ukPhonetic || ''

    // ■■■■■■■■■■■■■■■■■■■
    const definitions: string[] = []
```

```
    if (ec?.trs) {
      for (const tr of ec.trs) {
        if (tr.tr?.[0]?.l?.i) {
          const pos = tr.pos || '' // ■■
          const text = tr.tr[0].l.i[0] || ''
          definitions.push(pos ? `${pos} ${text}` : text)
        }
      }
    }
    const definition = definitions.join('■')

    // ■■■■■■■■
    const definitionsEn: string[] = []
    if (ee?.trs) {
      for (const tr of ee.trs) {
        if (tr.tr?.[0]?.l?.i) {
          definitionsEn.push(tr.tr[0].l.i)
        }
      }
    }
    const definition_en = definitionsEn.join('■')

    // ■■■■■■■■
    const partOfSpeech = ec?.trs?.[0]?.pos || syno?.pos || ''

    // ■■■■■■■■■■■
    const exampleSentences: { en: string; zh: string }[] = []
    if (blng && Array.isArray(blng)) {
      for (const pair of blng.slice(0, 5)) { // ■■5■■■
        const en = pair.sentence || pair['sentence-eng'] || ''
        const zh = pair['sentence-translation'] || ''
        if (en || zh) {
          exampleSentences.push({ en, zh })
        }
      }
    }

    const exampleSentence = exampleSentences.map(s => s.zh).join('\n')
    const exampleSentenceEn = exampleSentences.map(s => s.en).join('\n')

    // ■■■■■■
    const collocations: { en: string; zh: string }[] = []
    if (phrs && Array.isArray(phrs)) {
      for (const phr of phrs.slice(0, 5)) { // ■■5■■■
        const en = phr.phr?.headword?.l?.i || ''
        const zh = phr.phr?.trs?.[0]?.tr?.[0]?.l?.i || ''
        if (en || zh) {
          collocations.push({ en, zh })
        }
      }
```

```
    }

    const collocationEn = collocations.map(c => c.en).join('■')
    const collocation = collocations.map(c => c.zh).join('■')

    // ■■■■■■■■■■
    const synonyms: string[] = []
    if (syno && Array.isArray(syno)) {
      for (const synoItem of syno.slice(0, 5)) { // ■■5■■■■
        const pos = synoItem.pos || ''
        // ■ ■■■■■■■■■■■■ synoItem.syno ■■■
        const synoArray = synoItem.syno
        const words = Array.isArray(synoArray)
          ? synoArray.map((s: any) => s.word?.l?.i).filter(Boolean).join(', ')
          : (typeof synoArray === 'string' ? synoArray : '')
        if (words) {
          synonyms.push(pos ? `${pos} ${words}` : words)
        }
      }
    }

    // ■■■■■■■■■■■■■
    const forms: string[] = []
    if (simple?.wfs && Array.isArray(simple.wfs)) {
      for (const wf of simple.wfs) {
        const form = wf.wf?.name || ''
        const value = wf.wf?.value?.l?.i || ''
        if (form && value) {
          forms.push(`${form}: ${value}`)
        }
      }
    }

    return {
      word: word.trim(),
      phonetic: phonetic.replace(/\//g, ''),
      uk_phonetic: ukPhonetic.replace(/\//g, ''),
      us_phonetic: usPhonetic.replace(/\//g, ''),
      definition: definition,
      definition_en: definition_en,
      collocation: collocation,
      collocation_en: collocationEn,
      example_sentence: exampleSentence,
      example_sentence_en: exampleSentenceEn,
      part_of_speech: partOfSpeech,
      synonyms: synonyms.join('■'),
      forms: forms.join('■'),
      // ■■■■■■■■■■
      _raw_exampleSentences: exampleSentences,
      _raw_collocations: collocations,
```

```
      success: true
    }
  } catch (error) {
    // ████████████
    console.error('[parseYoudaoResponse] ████████████:', error)
    return {
      word: word.trim(),
      phonetic: '',
      uk_phonetic: '',
      us_phonetic: '',
      definition: '',
      definition_en: '',
      collocation: '',
      collocation_en: '',
      example_sentence: '',
      example_sentence_en: '',
      part_of_speech: '',
      synonyms: '',
      forms: '',
      _raw_exampleSentences: [],
      _raw_collocations: [],
      success: false
    }
  }
}


// ========== ██: src/lib/utils/text.ts ==========

/**
 * ██ HTML ██████████
 * @param text ██ HTML ██████
 * @returns ██████
 */
export function stripHtmlTags(text: string | null | undefined): string {
  if (!text) return ''
  return text.replace(/<[^>]*>/g, '')
}

/**
 * ██ HTML ██████ HTML ██
 * @param text ██ HTML █████████
 * @returns ██████
 */
export function decodeHtmlEntities(text: string | null | undefined): string {
  if (!text) return ''

  // ███ HTML ██
  let result = stripHtmlTags(text)

```

```
30698    // ■■■■ HTML ■■
30699    const entities: Record<string, string> = {
30700      '&lt;': '<',
30701      '&gt;': '>',
30702      '&amp;': '&',
30703      '&quot;': '"',
30704      '&apos;': "'",
30705      ' ': ' '
30706    }
30707
30708    result = result.replace(/&[a-zA-Z]+;/g, (entity) => entities[entity] || entity)
30709
30710    return result
30711  }
30712
30713
30714  // ========== ■■: src/lib/wordListUtils.ts ==========
30715
30716  /**
30717   * Word List Constants and Utilities
30718   *
30719   * ■■■■■■■■■■■■
30720   *
30721   * ■■■■
30722   * 1. ■■■■■■■■■■■■■■■■■■
30723   * 2. ■■■■■■■■■■■■■
30724   * 3. ■■■■■
30725   */
30726
30727  // ■■
30728  export const WORDS_PER_PAGE = 21
30729
30730  export const TIPS = [
30731    '■■■■■■■■■■',
30732    '■■■■■■■21■■■■■',
30733    '■■■■■■■■■■■■■■■',
30734    '■■■■■/■■/■■■■■■■■■',
30735    '■■■■■■■■■■■■■■■■■■',
30736    '■■■■■■■■■■■■■■■■'
30737  ]
30738
30739  // ■■■■■
30740  export type StatusFilter = 'all' | 'new' | 'known' | 'fuzzy' | 'unknown'
30741
30742  export type SortOrder = 'default' | 'random'
30743
30744  // ■■■■■■■
30745  export const STATUS_LABELS: Record<StatusFilter, string> = {
30746    'all': '■■',
30747    'new': '■■■',
```

```
  'known': '■■',
  'fuzzy': '■■',
  'unknown': '■■■'
}

// ■■■■■■■
export const STATUS_COLORS: Record<StatusFilter, string> = {
  'all': 'bg-slate-100 text-slate-700',
  'new': 'bg-blue-100 text-blue-700',
  'known': 'bg-green-100 text-green-700',
  'fuzzy': 'bg-yellow-100 text-yellow-700',
  'unknown': 'bg-red-100 text-red-700'
}

/**
 * ■■■■■■■■Fisher-Yates■■■
 * @param array - ■■■■■■■
 * @returns ■■■■■■■■
 */
export function shuffleArray<T>(array: T[]): T[] {
  const result = [...array]
  for (let i = result.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1))
    ;[result[i], result[j]] = [result[j], result[i]]
  }
  return result
}

/**
 * ■■■■■■■■■
 * @param status - ■■■■■■
 * @returns ■■■■■■■■
 */
export function getFilterLabel(status: StatusFilter): string {
  return STATUS_LABELS[status]
}

/**
 * ■■■■■■■
 * @param status - ■■■■■■
 * @returns ■■■Tailwind CSS■■
 */
export function getFilterColor(status: StatusFilter): string {
  return STATUS_COLORS[status]
}


// ========== ■■: src/lib/words-server.ts ==========

/**
```

```
 * ■■■■■■■■■■■
 *
 * ■■■■■■■■■■■■■■■■■■■■■■■■■■
 * ■■■■■■■■■■■■■■■■■■■■■
 */

import { createClient } from '@/lib/supabase/server'

// Word type definition (inline to avoid circular dependencies)
export interface Word {
  id: string
  word: string
  phonetic?: string
  uk_phonetic?: string
  us_phonetic?: string
  definition?: string
  definition_en?: string
  collocation?: string
  collocation_en?: string
  example_sentence?: string
  example_sentence_en?: string
  part_of_speech?: string
  chapter?: string
  chapter_id?: string | null  // ■ ■■■■ID■■
  theme?: string
  scene?: string
  status?: 'new' | 'unknown' | 'fuzzy' | 'known'
}

/**
 * ■■■■■■■■■■
 *
 * @param bookId - ■■ID
 * @param user - ■■■■■■■
 * @param page - ■■■■■1■
 * @param pageSize - ■■■■■■■■21■
 * @param status - ■■■■■ (all|unknown|fuzzy|known|new■■■all)
 * @param chapterId - ■■■■■■■■'all'■■■■■■■■
 * @returns ■■■■■■■■
 */
export async function getWordsForBookServer(
  bookId: string,
  user: any,
  page: number = 1,
  pageSize: number = 21,
  status: string = 'all',
  chapterId: string = 'all'
): Promise<{
  words: Word[]
  total: number
```

```
30848    count: number
30849    success: boolean
30850    error?: string
30851  }> {
30852    try {
30853      console.log(`■ [Server] Fetching words for book ${bookId}, page ${page}, status ${status}, chapter ${chapte
30854
30855      const supabase = await createClient()
30856
30857      // 1. ■■book■■■■■book■■
30858      const { data: book, error: bookError } = await supabase
30859        .from('books')
30860        .select('id, is_official, created_by, title, total_words')
30861        .eq('id', bookId)
30862        .single()
30863
30864      if (bookError || !book) {
30865        console.error('■ [Server] Book not found:', bookError)
30866        return {
30867          words: [],
30868          total: 0,
30869          count: 0,
30870          success: false,
30871          error: 'Book not found'
30872        }
30873      }
30874
30875      // 2. ■■■■
30876      if (book.is_official === false && book.created_by !== user.id) {
30877        console.error('■ [Server] Permission denied')
30878        return {
30879          words: [],
30880          total: 0,
30881          count: 0,
30882          success: false,
30883          error: 'Permission denied'
30884        }
30885      }
30886
30887      // 3. ■■■■■■■■■■status■
30888      const { data: progressData } = await supabase
30889        .from('word_progress')
30890        .select('word_id, status')
30891        .eq('user_id', user.id)
30892        .eq('book_id', bookId)
30893
30894      const statusMap = new Map<string, string>()
30895      progressData?.forEach((p: any) => {
30896        statusMap.set(p.word_id, p.status)
30897      })
```

```
30898
30899      // 4. ■■■■■■■■RPC■■
30900      const offset = (page - 1) * pageSize
30901
30902      let words: any[] | null = null
30903      let wordsError = null
30904
30905      // ■■ 'new' ■■■■■RPC■■■RPC■■■■■■■■■"■■■■■■■"■■■
30906      if (status !== 'new') {
30907        console.log('■ [Server] Trying optimized RPC...')
30908
30909        try {
30910          // ■■ ■■■■■■■■■■RPC■■hang■
30911          const timeoutPromise = new Promise((_, reject) =>
30912            setTimeout(() => reject(new Error('RPC timeout')), 5000) // 5■■■
30913          )
30914
30915          const result = await Promise.race([
30916            supabase.rpc('get_book_words_paginated_optimized', {
30917              book_uuid: bookId,
30918              offset_val: offset,
30919              limit_val: pageSize
30920            }),
30921            timeoutPromise
30922          ]) as any
30923
30924          words = result.data
30925          wordsError = result.error
30926
30927          if (!wordsError && words && words.length > 0) {
30928            console.log(`■ [Server] RPC returned ${words.length} words`)
30929
30930            // ■■status■■
30931            words = words.map((word: any) => ({
30932              ...word,
30933              status: statusMap.get(word.id) || 'new'
30934            }))
30935          }
30936        } catch (e: any) {
30937          console.log('■■ [Server] RPC exception:', e.message)
30938          // RPC■■■■■■■■■fallback
30939          wordsError = { message: e.message || 'RPC exception' }
30940        }
30941      }
30942
30943      // 5. Fallback: ■■■■■■
30944      if (wordsError || !words) {
30945        console.log('■ [Server] Using fallback query...')
30946
30947        // ■■chapters
```

```
30948        // ■ FIX: ■■■■■■
30949        const chaptersQuery = supabase
30950          .from('chapters')
30951          .select('id')
30952
30953        if (chapterId !== 'all') {
30954          chaptersQuery.eq('id', chapterId)
30955        } else {
30956          chaptersQuery.eq('book_id', bookId)
30957        }
30958
30959        const { data: chaptersData } = await chaptersQuery
30960
30961        if (!chaptersData) {
30962          return {
30963            words: [],
30964            total: book.total_words || 0,
30965            count: 0,
30966            success: true
30967          }
30968        }
30969
30970        const chapterIds = chaptersData.map((c: any) => c.id)
30971
30972        // ■■words
30973        const { data: fallbackWords, error: fallbackError } = await supabase
30974          .from('words')
30975          .select('id, word, phonetic, uk_phonetic, us_phonetic, definition, definition_en, collocation, collocati
30976          .in('chapter_id', chapterIds)
30977          .order('order_index', { ascending: true })
30978          .range(offset, offset + pageSize - 1)
30979
30980        if (fallbackError) {
30981          console.error('■ [Server] Fallback query failed:', fallbackError)
30982          return {
30983            words: [],
30984            total: book.total_words || 0,
30985            count: 0,
30986            success: false,
30987            error: 'Failed to fetch words'
30988          }
30989        }
30990
30991        // ■ ■■chapter_id■■■■■■■■■■■■chapter■■■■fallback■■■■■■■■■
30992        words = fallbackWords?.map((word: any) => ({
30993          ...word,
30994          status: statusMap.get(word.id) || 'new',
30995          chapter: '',  // fallback■■■■■■■■■■■■■■■
30996          theme: '',
30997          scene: ''
```

```
      })) || []

      console.log(`■ [Server] Fallback returned ${words.length} words`)
    }

    // 6. ■■■■■■■■■■status■■■'all'■
    let filteredWords = words || []
    let totalCount = book.total_words || 0

    if (status !== 'all' && words) {
      if (status === 'new') {
        const allProgressIds = new Set(progressData?.map((p: any) => p.word_id) || [])
        const newStatusIds = new Set(
          progressData
            ?.filter((p: any) => p.status === 'new')
            .map((p: any) => p.word_id) || []
        )

        filteredWords = words.filter((word: any) => {
          return !allProgressIds.has(word.id) || newStatusIds.has(word.id)
        })

        totalCount = book.total_words - allProgressIds.size + newStatusIds.size
      } else {
        filteredWords = words.filter((word: any) => word.status === status)
        totalCount = filteredWords.length
      }
    }

    console.log(`■ [Server] Returning ${filteredWords.length} words (total: ${totalCount})`)

    return {
      words: filteredWords as Word[],
      total: book.total_words || 0,
      count: totalCount,
      success: true
    }

  } catch (error: any) {
    console.error('■ [Server] Error:', error)
    return {
      words: [],
      total: 0,
      count: 0,
      success: false,
      error: error.message
    }
  }
}
```